

Sistema di Test per Ricarica DC

C# / .NET · WinUI 3 · MVVM · MES REST

ARCHITETTURA · QUALITÀ · ROADMAP

Marcomattia Mocellin

Senior Software Engineer · Technical Lead · Software Architect



Un profilo costruito tra software industriale e integrazione

PROFESSIONAL OVERVIEW

Senior Software Engineer e Technical Lead con **oltre 10 anni** di esperienza in software industriale, sistemi embedded, elettronica, metrologia e Industry 4.0.

Competenze rilevanti per il case study:

- Architettura software C# / .NET e principi SOLID
- MES, REST API, CAN Bus, MQTT e connettività industriale
- Coordinamento tra software, hardware, firmware e validazione sul campo

10+ anni

software industriale · embedded · metrologia

C# / .NET

architettura
e modernizzazione

Industry 4.0

piattaforme
e integrazione

Approccio di lavoro

dalla specifica alla validazione sul campo

Esperienza trasferibile al contesto di un test system

2016 – 2020

2020 – oggi

FPT Industrie · Test / Software Engineer

- Soluzioni di misura, CNC e ambienti manifatturieri Industry 4.0
- Integrazioni tra MES, pianificazione e controlli numerici
- Qualifica macchina, testing e supporto alla messa in servizio

voestalpine Böhler Welding Arc Technology

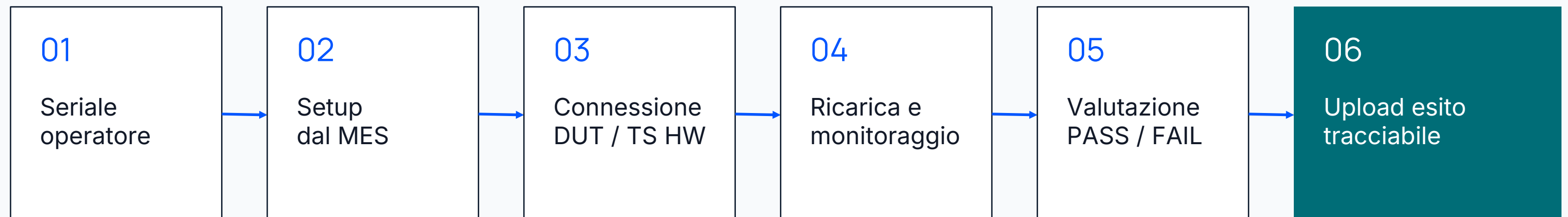
- Technical Coordinator per piattaforma Industry 4.0 di Gruppo dal 2022
- Project Manager per una nuova interfaccia hardware dal 2022
- AI Tech Lead per iniziative di Gruppo dal 2025

Risultati rilevanti: 5 brevetti internazionali, sviluppo di un sistema di misura basato su laser a bordo macchina con risoluzione entro 10 µm, modernizzazione della suite Weld@Net in C# e miglioramenti di performance dichiarati fino al 1200% in componenti C++ selezionati.



Dal seriale al risultato tracciabile

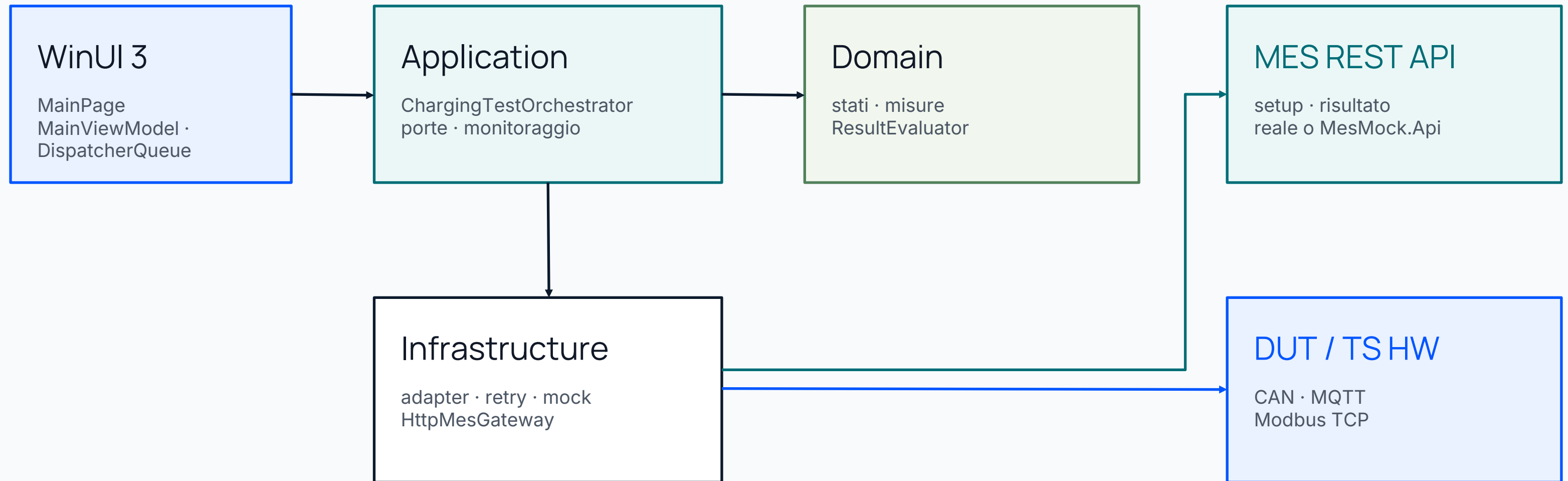
Un progetto eseguibile per dimostrare separazione delle responsabilità, workflow sicuro e integrazione esterna.



Confine del progetto

Dimostra il comportamento della Test System App. Il mock MES rende osservabile l'API, ma non pretende di sostituire il MES produttivo.

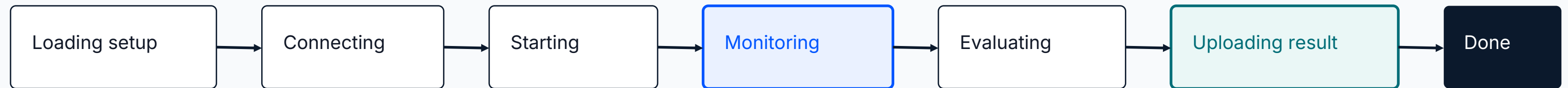
Separare ciò che cambia



Principio guida: Interfaccia in WinUI, regole pure nel Domain, orchestrazione nell'Application, dettagli volatili nell'Infrastructure.

Concorrente, ma deterministica nel punto che conta

MACCHINA A STATI DELLA SESSIONE



Due stream, un monitor verificabile

- DUT e hardware di test producono `IAsyncEnumerable<MeterReading>`
- `ReadingMonitor` li fa convergere in un `Channel`
- La UI riceve progress aggiornato senza bloccare il thread grafico

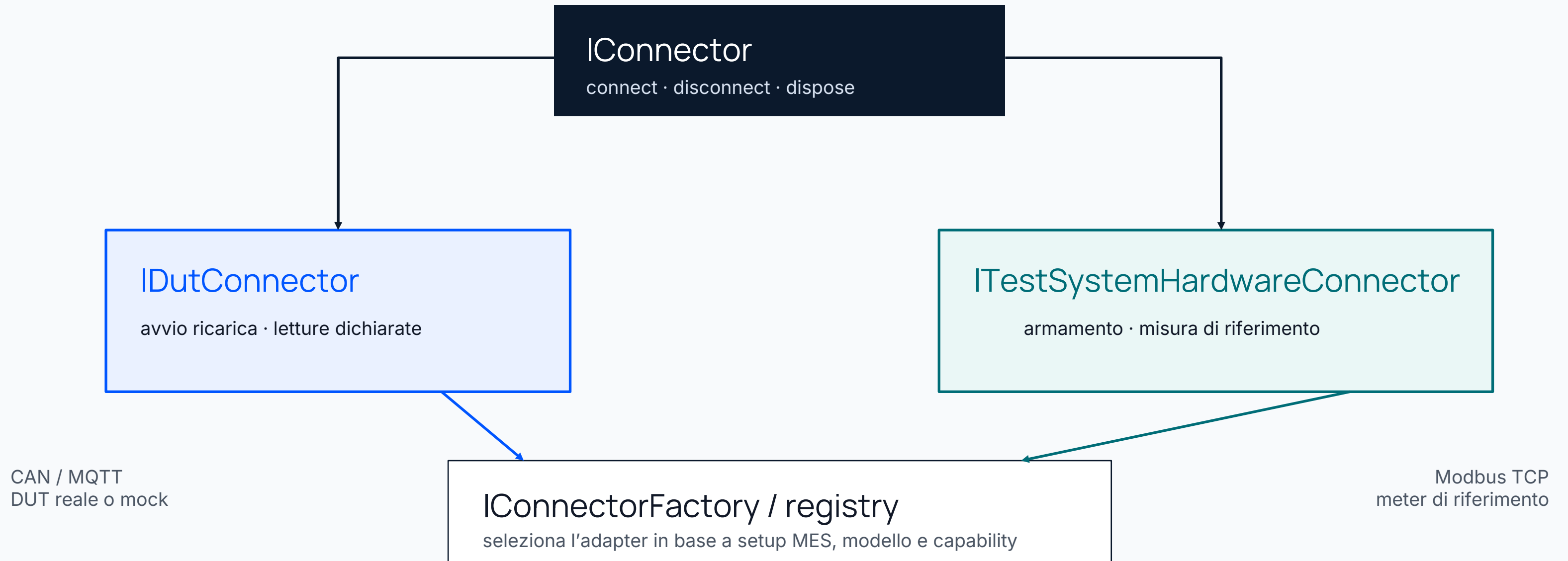
INVARIANTE DI QUALITÀ

Il risultato finale richiede campioni correlati.

Gli ultimi valori devono condividere la stessa **Sequence**; altrimenti il risultato non è dimostrabile.

Capability prima dei protocolli

Il caso d'uso chiede cosa un dispositivo sa fare, non come inviare un frame a uno specifico vendor.



MES REST: contratto esplicito e dimostrabile

Il case study richiede una connessione al MES via API; lo skeleton rende il confine HTTP verificabile senza affermare di implementare il MES reale.

HTTP RESOURCES

GET

`/api/test-sessions/{serialNumber}/setup`

recupera specifica, tolleranze e configurazione dei connettori

PUT

`/api/test-sessions/{sessionId}/result`

salva l'esito verificabile della sessione

Separazione del confine REST

Mes.Contracts
DTO JSON

HttpMesGateway
mapper + client

MesMock.Api · Minimal API locale
demo manuale e test end-to-end

Una demo utile anche per chi opera e per chi la evolve

TEST SYSTEM — CHARGING SESSION

IT · EN · DE · JA

NUMERO DI SERIE

DEMO-0001

AVVIA

DUT METER

50.00 kW

REFERENCE METER

50.00 kW

DIAGNOSTICA · STATO ·
DETTAGLIO ESITO

MVVM + async

binding, comandi asincroni e DispatcherQueue proteggono la responsività della UI.

Localizzazione

italiano, inglese, tedesco e giapponese per validare Unicode e sostituibilità del provider.

Scenari demo

bottoni dedicati per test MES assente e letture divergenti, sempre isolati dal MES configurato.

Test mirati

15 test su regole, orchestrazione, localizzazione e contratto REST del client.



Il progetto è intenzionalmente onesto sui limiti di produzione

Ogni rischio rilevante viene mantenuto visibile e tradotto in un prossimo passo architetturale.

CRITICITÀ	LIMITAZIONE NELLA DEMO	MITIGAZIONE IN PRODUZIONE
Adapter vendor-specific CAN / Modbus / MQTT	mock e contratti; nessun driver vendor reale nello skeleton	adapter certificati, timeout classificati, prove su banco
Affidabilità upload MES rete indisponibile	PUT idempotente e mock RAM; nessuna persistenza durevole	outbox locale, retry classificato, correlation e idempotency key
Qualità dei campioni sincronizzazione fisica	invariante Sequence; prova semplificata sul campione finale	skew temporale massimo, barriera di fine test, quality checks
Sicurezza e osservabilità accesso, audit, diagnosi	nessuna autenticazione o telemetria centralizzata nel mock	OAuth2 / credenziali protette, log strutturati, metriche e audit trail

Scelta chiave: i limiti non vengono nascosti nel core; sono confinati agli adapter e documentati come backlog di produzione.

Grazie dell'attenzione.

FASE 1

Integrare



Adapter CAN / Modbus /
MQTT reali, capability
discovery e test di
integrazione su banco.

FASE 2

Irrobustire



Outbox, autenticazione MES,
configurazione protetta,
correlation ID e telemetria.

FASE 3

Validare



Prove con hardware, fault
injection, load test e criteri di
accettazione operatore.

Marcomattia Mocellin · Senior Software Engineer / Technical Lead · marcomattia.mocellin@gmail.com